

# Asynchronous FIFO

DUT Specification Sheet | Dual-Clock Gray-Code Synchronized FIFO

| Doc Rev | Status        | Top Module | Target                                     |
|---------|---------------|------------|--------------------------------------------|
| 1.0     | Design Review | async_fifo | FPGA / ASIC, synchronous-design dual clock |

## 1. Overview

The `async_fifo` is a parameterizable, dual-clock-domain First-In-First-Out buffer intended for safe data transfer between two independently clocked agents with no fixed phase relationship. The design implements the standard Gray-code pointer / two-flop synchronizer architecture (Cummings, "Simulation and Synthesis Techniques for Asynchronous FIFO Design") to guarantee functionally correct full and empty flag generation across the clock domain crossing (CDC) boundary.

The read interface is implemented as combinational (first-word-fall-through / show-ahead) data output: the entry addressed by the current read pointer is presented on `o_rd_data` whenever the FIFO is not empty, without requiring a registered read latency cycle.

## 2. Key Features

- Fully synchronous design within each clock domain; only Gray-coded pointers cross domains
- Two-flip-flop synchronizer per direction, instantiated with domain-local clock and reset
- Look-ahead (next-state) FULL / EMPTY flag generation -- no combinational glitch on flag outputs
- Gray-to-binary-conversion-free full/empty comparison (direct Gray-domain compare)
- Parameterizable DEPTH and DATA\_WIDTH
- First-Word-Fall-Through (FWFT) read data path
- Independent, domain-local active-low asynchronous resets (`i_wr_rstn`, `i_rd_rstn`)

## 3. Architecture / Module Hierarchy

The design is partitioned into five RTL modules. The top level instantiates write-side and read-side pointer handlers, one synchronizer per pointer-crossing direction, and the dual-port memory array.

| Module                      | Role                                                         | Clock Domain          |
|-----------------------------|--------------------------------------------------------------|-----------------------|
| <code>async_fifo</code>     | Top level: structural integration of all sub-blocks          | Both (boundary)       |
| <code>wr_ptr_handler</code> | Binary/Gray write-pointer generation, next-state FULL flag   | <code>i_wr_clk</code> |
| <code>rd_ptr_handler</code> | Binary/Gray read-pointer generation, next-state EMPTY flag   | <code>i_rd_clk</code> |
| <code>ff_sync</code>        | 2-stage flip-flop synchronizer (Gray pointer CDC crossing)   | Destination clock     |
| <code>fifo_mem</code>       | Dual-port memory array; registered write, combinational read | Both (split ports)    |

Two `ff_sync` instances are used: `wr_ff` synchronizes the read-side Gray pointer into the write clock domain (for FULL generation); `rd_ff` synchronizes the write-side Gray pointer into the read clock domain (for EMPTY generation).

## 4. Parameters

| Parameter  | Default                              | Description                                                                                                                                                                      |
|------------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DEPTH      | 8                                    | Number of storage locations. Must be a power of two -- the Gray-code wrap and look-ahead full/empty comparison logic assumes $\text{PTR\_WIDTH} = \log_2(\text{DEPTH})$ exactly. |
| DATA_WIDTH | 4                                    | Width of each storage word, in bits.                                                                                                                                             |
| PTR_WIDTH  | $\lceil \log_2(\text{DEPTH}) \rceil$ | Derived (localparam). Pointers are $\text{PTR\_WIDTH}+1$ bits wide; the extra MSB resolves the full-vs-empty ambiguity when read and write pointers are numerically equal.       |

## 5. Top-Level Port List -- async\_fifo

### Write Clock Domain

| Port      | Dir | Width      | Description                                                                            |
|-----------|-----|------------|----------------------------------------------------------------------------------------|
| i_wr_clk  | in  | 1          | Write-domain clock                                                                     |
| i_wr_rstn | in  | 1          | Write-domain asynchronous active-low reset                                             |
| i_wr_en   | in  | 1          | Write enable / push request                                                            |
| i_wr_data | in  | DATA_WIDTH | Write data                                                                             |
| o_full    | out | 1          | FIFO full flag (registered, look-ahead). Writes are internally blocked while asserted. |

### Read Clock Domain

| Port      | Dir | Width      | Description                                                                            |
|-----------|-----|------------|----------------------------------------------------------------------------------------|
| i_rd_clk  | in  | 1          | Read-domain clock                                                                      |
| i_rd_rstn | in  | 1          | Read-domain asynchronous active-low reset                                              |
| i_rd_en   | in  | 1          | Read enable / pop request                                                              |
| o_rd_data | out | DATA_WIDTH | Read data (combinational / FWFT -- valid whenever o_empty is low)                      |
| o_empty   | out | 1          | FIFO empty flag (registered, look-ahead). Reads are internally blocked while asserted. |

## 6. Functional Description

### 6.1 Pointer Representation

Both write and read pointers are maintained in two forms: a binary form (used to address the memory array) and a Gray-coded form (used for the cross-domain comparison). Each pointer is  $\text{PTR\_WIDTH}+1$  bits wide; the lower  $\text{PTR\_WIDTH}$  bits address the memory, and the additional MSB tracks wrap-around so that the full and empty conditions -- both of which occur when read and write addresses are numerically equal -- can be distinguished.

### 6.2 Full Flag Generation (write domain)

FULL is asserted when the next write Gray pointer equals the synchronized read Gray pointer with its two MSBs inverted -- the standard wrap-detection condition for Gray-coded pointers, requiring no Gray-to-binary conversion. The comparison uses the next-state pointer value, so the flag updates on the same edge the final write into a full FIFO would occur, eliminating the one-cycle lag a purely registered comparison would introduce.

### 6.3 Empty Flag Generation (read domain)

EMPTY is asserted when the next read Gray pointer equals the synchronized write Gray pointer directly. As with FULL, the comparison is performed on the next-state pointer for look-ahead timing.

### 6.4 Clock Domain Crossing

The write pointer (Gray) is synchronized into the read domain, and the read pointer (Gray) is synchronized into the write domain, each through an independent 2-flop synchronizer clocked and reset by the destination domain. Because the pointers are Gray-coded, only a single bit toggles per increment, bounding the synchronizer's worst-case metastability outcome to a one-count error rather than a multi-bit corrupted value.

### 6.5 Memory Array

fifo\_mem implements a simple dual-port array (reg [DATA\_WIDTH-1:0] mem [0:DEPTH-1]). Writes are registered on i\_wr\_clk and qualified by i\_wr\_en & ~i\_full. The read data path is combinational (assign o\_rd\_data = mem[RD\_ADDR]) -- i.e. FWFT/show-ahead behavior: valid data for the current head entry is present on o\_rd\_data continuously whenever o\_empty is low, without needing i\_rd\_en to be asserted first.

## 7. Reset Behavior

| Signal                  | Type                                  | State on Reset |
|-------------------------|---------------------------------------|----------------|
| o_b_wr_ptr / o_g_wr_ptr | Async, active-low (i_wr_rstn)         | 0              |
| o_full                  | Async, active-low (i_wr_rstn)         | 0 (not full)   |
| o_b_rd_ptr / o_g_rd_ptr | Async, active-low (i_rd_rstn)         | 0              |
| o_empty                 | Async, active-low (i_rd_rstn)         | 1 (empty)      |
| ff_sync_stages (f1, f2) | Async, active-low (domain-local rstn) | 0              |

Note: i\_wr\_rstn and i\_rd\_rstn are independent. Each synchronizer is reset by its own destination-domain reset only, which avoids introducing an asynchronous reset-removal CDC hazard between domains.

## 8. Timing Characteristics

| Parameter             | Value                      | Notes                                            |
|-----------------------|----------------------------|--------------------------------------------------|
| Write-to-full latency | 0 cycles                   | Look-ahead flag; FULL reflects the pending write |
| Read-to-empty latency | 0 cycles                   | Look-ahead flag; EMPTY reflects the pending read |
| Write data to memory  | 1 i_wr_clk edge            | Registered write port                            |
| Memory to o_rd_data   | Combinational              | FWFT; no registered read latency                 |
| Pointer CDC latency   | 2 destination-clock cycles | Per 2-flop synchronizer stage                    |
| Max throughput        | 1 word / clock             | Per domain, while not full / not empty           |

## 9. Verification Considerations

The following checks are recommended for a constrained-random / UVM environment targeting this DUT, in addition to standard data-integrity and ordering scoreboard checks:

- Elaboration-time check that DEPTH is a power of two (PTR\_WIDTH consistency)
- No-overflow / no-underflow assertions: flag (don't just silently drop) writes while o\_full or reads while o\_empty
- Gray-code one-hot transition assertion on o\_g\_wr\_ptr and o\_g\_rd\_ptr -- at most one bit may change per active clock edge
- Full/empty mutual exclusivity check (o\_full and o\_empty must never assert simultaneously)
- Independent reset sequencing: verify correct behavior under skewed / independent assertion and de-assertion of i\_wr\_rstn and i\_rd\_rstn
- Pointer wrap-around coverage: drive write/read activity through at least two full address wraps
- Back-to-back full-to-not-full and empty-to-not-empty transition coverage at varying clock ratios (wr\_clk faster, slower, and asynchronous to rd\_clk)

## 10. Revision History

| Rev | Date       | Description                                             |
|-----|------------|---------------------------------------------------------|
| 1.0 | 2026-06-30 | Initial specification, generated from RTL design review |

This specification was derived from the provided RTL source (async\_fifo.v, wr\_ptr\_handler.v, rd\_ptr\_handler.v, ff\_sync.v, fifo\_mem.v) and reflects design intent as implemented; it does not represent simulation- or silicon-validated electrical timing.